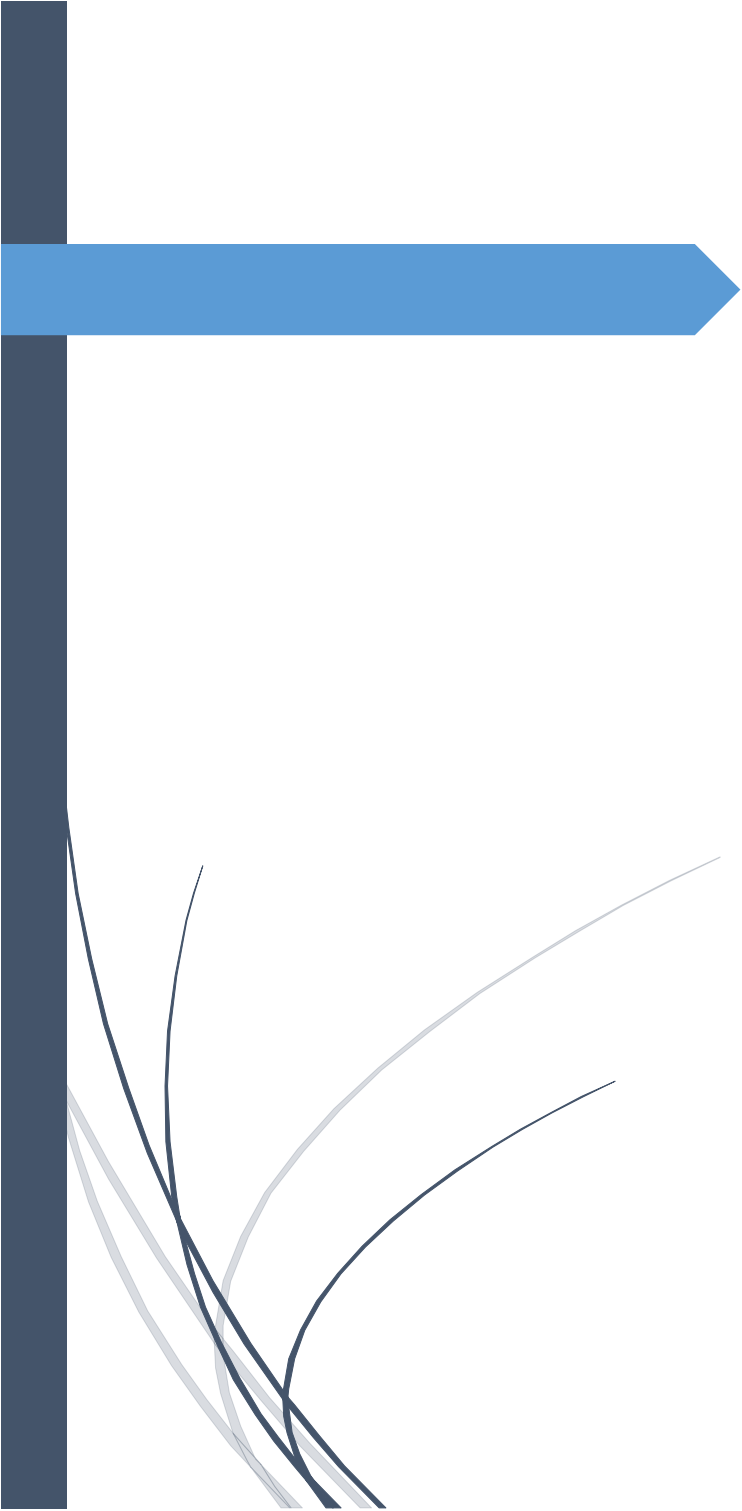




Προπτυχιακό Μάθημα: «Μεταφραστές» Προγραμματιστική Εργασία Εξαμήνου

Πανεπιστήμιο Ιωαννίνων Πολυτεχνική Σχολή
Τμήμα Μηχανικών Η/Υ και Πληροφορικής

Ακ. Έτος: 2024-25

Μέλη ομάδας

Γεώργιος Σιώζος, 5434

Δημήτριος Καλτσογιάννης, 5067

Περιεχόμενα

<u>Ενότητα 1^η</u> : Περιγραφή Compiler για την Γλώσσα Greek++.....	2
<u>Ενότητα 2^η</u> : Τρόπος Λειτουργίας του Compiler.....	4
2.1 Λεκτική Ανάλυση.....	4
2.2 Συντακτική Ανάλυση.....	8
2.3 Ενδιάμεσος Κώδικας.....	13
2.4 Πίνακας Συμβόλων.....	16
<u>Ενότητα 3^η</u> : Παραγωγή Τελικού Κώδικα.....	20

Ενότητα 1^η: Εισαγωγή στον Compiler Greek++

Ο μεταγλωττιστής Greek++ που υλοποιήθηκε έχει σχεδιαστεί σύμφωνα με τη δομή ενός παραδοσιακού compiler τεσσάρων φάσεων, ακολουθώντας την κλασική αρχιτεκτονική που συναντάμε στη θεωρία μεταφραστών. Ο στόχος του είναι να μεταφράζει προγράμματα γραμμένα στη γλώσσα Greek++ σε ενδιάμεσο κώδικα και τελικά σε RISC-V Assembly, διατηρώντας παράλληλα συμβατότητα με τα συντακτικά και σημασιολογικά χαρακτηριστικά της γλώσσας.

Η λειτουργία του compiler είναι αυστηρά δομημένη σε στάδια, καθένα εκ των οποίων είναι υπεύθυνο για μια διακριτή λειτουργία της διαδικασίας μεταγλώττισης. Αυτή η πολυεπίπεδη ανάλυση επιτρέπει τη σταδιακή μετάβαση από τον πηγαίο κώδικα σε εντολές χαμηλού επιπέδου, με έλεγχο συντακτικών και σημασιολογικών λαθών, καθώς και με ενσωμάτωση μηχανισμών όπως πίνακας συμβόλων, διαχείριση scope, και παραγωγή τετραπλέτων.

Τα βασικά στάδια περιλαμβάνουν:

- **Λεξική Ανάλυση (Lexical Analysis)**: Εντοπισμός των βασικών μονάδων (tokens) του κώδικα.
- **Συντακτική Ανάλυση (Parsing)**: Δομική ερμηνεία των tokens βάσει της γραμματικής της γλώσσας.
- **Παραγωγή Ενδιάμεσου Κώδικα (Intermediate Code Generation)**: Δημιουργία τετραπλέτων για κάθε πράξη.
- **Σημασιολογική Ανάλυση (Semantic Analysis)**: Διαχείριση μεταβλητών μέσω πίνακα συμβόλων (Symbol Table).
- **Γεννήτρια Κώδικα (Code Generation)**: Παραγωγή τελικού κώδικα σε RISC-V Assembly.

Ενότητα 2^η: Τρόπος Λειτουργίας του Compiler

Ο μεταγλωττιστής που υλοποιήσαμε έχει ως στόχο τη μετάφραση προγραμμάτων γραμμένων στην γλώσσα προγραμματισμού **Greek++** σε κώδικα **Assembly** για την αρχιτεκτονική **RISC-V**. Η λειτουργία του βασίζεται σε μια διαδοχική αλυσίδα σταδίων, που εξασφαλίζουν την ορθή ανάλυση, επεξεργασία και τελική παραγωγή εκτελέσιμου κώδικα. Αρχικά, το πρόγραμμα εισόδου διαβάζεται και περνά από τον λεκτικό αναλυτή, ο οποίος το διασπά σε λεκτικές μονάδες (**tokens**). Στη συνέχεια, ο συντακτικός αναλυτής ελέγχει αν η ακολουθία των **tokens** ακολουθεί τους κανόνες της γλώσσας και δημιουργεί μια ενδιάμεση αναπαράσταση του προγράμματος. Παράλληλα, ο πίνακας συμβόλων διαχειρίζεται τις μεταβλητές και τα αναγνωριστικά, διασφαλίζοντας τη σωστή χρήση τους και αποτρέποντας συντακτικά ή σημασιολογικά σφάλματα. Ακολουθεί η παραγωγή ενδιάμεσου κώδικα με τη μορφή τετραδών, που λειτουργεί ως γέφυρα μεταξύ της συντακτικής ανάλυσης και της τελικής μετάφρασης. Τέλος, ο ενδιάμεσος κώδικας μετατρέπεται σε **Assembly**.

2.1 Λεκτική Ανάλυση

Στην κλάση **Lexer** υλοποιείται το στάδιο της λεξικής ανάλυσης του μεταγλωττιστή. Ο ρόλος της είναι να μετατρέπει τον πηγαίο κώδικα της γλώσσας Greek++ σε μια ακολουθία από **tokens**, δηλαδή τις βασικές λεκτικές μονάδες που θα χρησιμοποιηθούν στα επόμενα στάδια της μεταγλώττισης. Κατά την αρχικοποίησή της, η κλάση δέχεται ως είσοδο τον πηγαίο κώδικα του προγράμματος.

Η κύρια μέθοδος **tokenize** επεξεργάζεται το κείμενο, το διαχωρίζει σε λέξεις και σύμβολα με τη βοήθεια αντικαταστάσεων (π.χ. προσθήκη κενών γύρω από παρενθέσεις, άγκιστρα, ερωτηματικά), και στη συνέχεια ταξινομεί κάθε στοιχείο ως λέξη-κλειδί, τελεστή, αριθμό, ταυτοποιητή ή διαχωριστικό, με βάση προκαθορισμένα σύνολα (**KEYWORDS, OPERATORS, DELIMITERS**). Για κάθε αναγνωρισμένη μονάδα δημιουργείται ένα αντικείμενο **Token**, ενώ στο τέλος προστίθεται πάντα ένα ειδικό **token** τέλους αρχείου (**EOF**). Η υλοποίηση αυτή διασφαλίζει ότι η είσοδος μετατρέπεται σε μια δομημένη και ευανάγνωστη μορφή για τον συντακτικό αναλυτή.

Σημαντικές εντολές και μέθοδοι της κλάσης Lexer:

<code>__init__(self, code)</code>	Αρχικοποιεί τον Lexer με τον πηγαίο κώδικα που θα αναλυθεί.
<code>tokenize(self)</code>	Επεξεργασία του κειμένου με αντικαταστάσεις χαρακτήρων για σωστό διαχωρισμό συμβόλων. Επανάληψη σε κάθε λέξη και ταξινόμηση της ως KEYWORD, OPERATOR, DELIMITER, NUMBER ή IDENTIFIER. Δημιουργία αντικειμένων Token για κάθε αναγνωρισμένη λεκτική μονάδα. Επιστροφή της τελικής λίστας tokens, μαζί με το ειδικό token EOF.

Τα σύνολα **KEYWORDS**, **OPERATORS**, **DELIMITERS** ορίζονται ως μεταβλητές κλάσης και περιλαμβάνουν όλες τις λέξεις-κλειδιά, τελεστές και διαχωριστικά της γλώσσας.

Η κλάση **Token** χρησιμοποιείται για την αναπαράσταση κάθε λεκτικής μονάδας με τον τύπο και την τιμή της.

2.2 Συντακτική Ανάλυση

Ο συντακτικός αναλυτής (Parser) αποτελεί το δεύτερο βασικό στάδιο της υλοποίησης του μεταγλωττιστή και έχει ως κύριο στόχο τη μετατροπή της ακολουθίας των λεκτικών μονάδων (tokens), που παράγονται από τον λεκτικό αναλυτή, σε μια δομημένη αναπαράσταση του προγράμματος σύμφωνα με τους συντακτικούς κανόνες της γλώσσας. Η διαδικασία αυτή εξασφαλίζει ότι το πρόγραμμα είναι συντακτικά ορθό και παράγει ενδιάμεσο κώδικα, ο οποίος θα χρησιμοποιηθεί στα επόμενα στάδια της μεταγλώττισης.

Η κλάση **Parser** υλοποιεί έναν αναλυτή αναδρομικής λειτουργίας και βασίζεται σε μια σειρά από μεθόδους που αντιστοιχούν σε συντακτικούς κανόνες της γλώσσας. Κατά την αρχικοποίηση της κλάσης, ο **Parser** λαμβάνει ως είσοδο τη λίστα των **tokens**, τον ενδιάμεσο κώδικα και τον πίνακα συμβόλων. Η ανάλυση ξεκινά με τη μέθοδο **parse_program**, η οποία διαχειρίζεται το άνοιγμα ενός νέου **scope** στον πίνακα συμβόλων και αναγνωρίζει τη βασική δομή του προγράμματος, ελέγχοντας την ύπαρξη της λέξης-κλειδί "πρόγραμμα", το όνομα του προγράμματος, τα απαραίτητα σύμβολα ομαδοποίησης και το μπλοκ εντολών. Η μέθοδος αυτή καλεί

τη **parse_programblock**, η οποία με τη σειρά της διαχωρίζει και αναλύει τις δηλώσεις μεταβλητών (μέσω της **parse_declarations**) και την ακολουθία των εντολών (μέσω της **parse_sequence**). Οι δηλώσεις μεταβλητών αναλύονται από τη **parse_declarations**, η οποία εντοπίζει τη λέξη-κλειδί "δήλωση" και καλεί τη **parse_varlist** για την αναγνώριση και δήλωση των μεταβλητών στον πίνακα συμβόλων. Η διαδικασία αυτή διασφαλίζει ότι κάθε μεταβλητή δηλώνεται μόνο μία φορά και αποδίδεται σε αυτήν μοναδική διεύθυνση μνήμης.

Η ανάλυση της ακολουθίας εντολών πραγματοποιείται από τη **parse_sequence**, η οποία διαβάζει διαδοχικά τις εντολές του προγράμματος μέχρι να εντοπιστεί το τέλος του μπλοκ ή κάποια λέξη-κλειδί που σηματοδοτεί αλλαγή ροής, όπως "**εάν_τέλος**", "**όσο_τέλος**" ή "**αλλιώς**". Για κάθε εντολή, καλείται η **parse_statement**, η οποία αναγνωρίζει το είδος της εντολής και καλεί την αντίστοιχη μέθοδο ανάλυσης. Ειδικότερα, οι εκχωρήσεις τιμών σε μεταβλητές αναλύονται με τη **parse_assignment**, όπου ελέγχεται η ύπαρξη της μεταβλητής στον πίνακα συμβόλων, αναλύεται η έκφραση μέσω της **parse_expression** και παράγεται η αντίστοιχη εντολή στον ενδιαμέσο κώδικα. Οι εκφράσεις και οι όροι αναλύονται με τις **parse_expression** και **parse_term** αντίστοιχα, με υποστήριξη για αριθμητικές

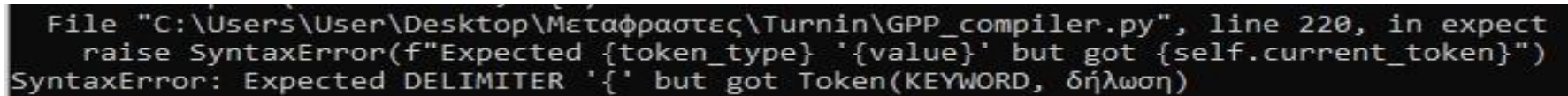
πράξεις και παρενθέσεις, ενώ παράλληλα γίνεται έλεγχος ύπαρξης μεταβλητών στον πίνακα συμβόλων.

Στην συνέχεια, η ανάλυση των δομών ελέγχου πραγματοποιείται μέσω των μεθόδων **parse_if** και **parse_while**, οι οποίες διαχειρίζονται τα αντίστοιχα μπλοκ εντολών και καλούν τη `parse_condition` για την ανάλυση των συνθηκών. Η **parse_condition** αναλύει συγκριτικές εκφράσεις, παράγει τον σχετικό ενδιάμεσο κώδικα σύγκρισης και εξασφαλίζει τη σωστή ροή εκτέλεσης του προγράμματος. Επιπλέον, οι εντολές εισόδου και εξόδου αναλύονται με τις **parse_input** και **parse_print**, όπου ελέγχεται η ύπαρξη της μεταβλητής ή της έκφρασης και παράγεται η αντίστοιχη εντολή στον ενδιάμεσο κώδικα.

Επίσης, η μέθοδος **advance** μετακινεί τον δείκτη στο επόμενο **token**, ενώ η **expect** ελέγχει αν το τρέχον **token** είναι του αναμενόμενου τύπου και τιμής, επιτρέποντας την έγκαιρη ανίχνευση συντακτικών σφαλμάτων και τη διακοπή της εκτέλεσης με κατάλληλο μήνυμα σε περίπτωση λάθους. Καθ' όλη τη διάρκεια της

ανάλυσης, ο **Parser** συνεργάζεται στενά με τον πίνακα συμβόλων, ώστε να διασφαλίζεται η ορθότητα των δηλώσεων και της χρήσης των μεταβλητών.

Συνολικά, η κλάση **Parser** διασφαλίζει τη συντακτική ορθότητα του προγράμματος, παράγει ενδιάμεσο κώδικα τριών διευθύνσεων και προετοιμάζει τα δεδομένα για τα επόμενα στάδια της μεταγλώττισης, παρέχοντας σαφή και άμεσα μηνύματα σε περίπτωση συντακτικών λαθών. Όπως παρουσιάζεται στην εικόνα 1.

A screenshot of a terminal window showing a Python SyntaxError. The text is as follows:

```
File "C:\Users\User\Desktop\Μεταφραστές\Turnin\GPP_compiler.py", line 220, in expect
    raise SyntaxError(f"Expected {token_type} '{value}' but got {self.current_token}")
SyntaxError: Expected DELIMITER '{' but got Token(KEYWORD, δήλωση)
```

Εικόνα 1

Διαχείριση σφαλμάτων από τον compiler

Η μέθοδος **expect(self, token_type, value=None)** είναι υπεύθυνη για τον έλεγχο αν το τρέχον **token** είναι του αναμενόμενου τύπου και, αν απαιτείται, της αναμενόμενης

τιμής. Εάν το **token** δεν αντιστοιχεί στις προσδοκίες του συντακτικού αναλυτή, τότε εκτελείται η υπογραμμισμένη εντολή της εικόνας 2:

```
def expect(self, token_type, value=None):  
    if self.current_token.type != token_type or (value and self.current_token.value != value):  
        raise SyntaxError(f"Expected {token_type} '{value}' but got {self.current_token}")  
    self.advance()
```

Εικόνα 2

Με αυτόν τον τρόπο, διακόπτεται άμεσα η εκτέλεση και εμφανίζεται ένα σαφές μήνυμα λάθους που περιγράφει τι αναμενόταν και τι βρέθηκε στη θέση αυτή του κώδικα. Αυτό το μοτίβο ακολουθείται σε όλα τα σημεία όπου απαιτείται συγκεκριμένη ακολουθία **tokens**, όπως για παράδειγμα στην αρχή ενός προγράμματος, στη δήλωση μεταβλητών, σε εκχωρήσεις τιμών, σε εκφράσεις και σε δομές ελέγχου.

2.3 Ενδιάμεσος Κώδικας

Η παραγωγή του ενδιάμεσου κώδικα αποτελεί ένα από τα σημαντικότερα στάδια της διαδικασίας μεταγλώττισης. Σε αυτή τη φάση, ο συντακτικός αναλυτής, καθώς επεξεργάζεται το πρόγραμμα, μετατρέπει τις διάφορες εντολές και εκφράσεις σε μια ενδιάμεση μορφή που ονομάζεται «**τετράδες**» (ή τριάδες/τριπλέτες), γνωστή και ως κώδικας τριών διευθύνσεων (three-address code). Ο ενδιάμεσος αυτός κώδικας είναι ανεξάρτητος από τη γλώσσα στόχο και έχει ως στόχο να απλοποιήσει τη μετάβαση από τη δομή του αρχικού προγράμματος στη μορφή που απαιτεί η τελική μηχανή.

Στην παρούσα υλοποίηση, ο ενδιάμεσος κώδικας παράγεται μέσω της κλάσης **IntermediateCode**, η οποία διατηρεί μια λίστα από τετράδες. Κάθε τετράδα αποτελείται από τέσσερα πεδία: τον **τελεστή** (op), τα δύο **ορίσματα** (arg1, arg2) και το **αποτέλεσμα** (res). Για παράδειγμα, μια εκχώρηση τιμής σε μεταβλητή αποτυπώνεται ως τετράδα της μορφής (**:=, expr_result, _, id_name**), ενώ μια αριθμητική πράξη ως (**+', left, right, temp**), όπου το **temp** είναι μια προσωρινή μεταβλητή που δημιουργείται αυτόματα για την αποθήκευση ενδιάμεσων αποτελεσμάτων.

Επίσης, Η μέθοδος **print_code** της κλάσης **IntermediateCode** είναι υπεύθυνη για την απεικόνιση του ενδιάμεσου κώδικα που έχει παραχθεί κατά τη διάρκεια της συντακτικής ανάλυσης. Πιο συγκεκριμένα, η μέθοδος αυτή διατρέχει διαδοχικά τη λίστα των τετράδων που έχουν αποθηκευτεί στη μεταβλητή **instructions** και εκτυπώνει κάθε μία από αυτές με έναν ευανάγνωστο και αριθμημένο τρόπο. Παρακάτω απεικονίζεται ο ενδιάμεσος κώδικας ενός Greek++ προγράμματος.

πρόγραμμα program() {	1: begin_block main __
δήλωση x, y, z;	2: := 6 _ x
x := 6;	3: := 1 _ y
y := 1;	4: := 0 _ z
z := 0;	5: > x 0 T_0
όσο x > 0 επανάλαβε	6: + z y T_1
z := z + y;	7: := T_1 _ z
x := x - 1;	8: - x 1 T_2
εάν z > 10 τότε	9: := T_2 _ x
γράψε z;	10: > z 10 T_3
αλλιώς	11: write z __
γράψε y;	12: write y __
εάν_τέλος;	13: write z __
όσο_τέλος;	14: end_block main __
γράψε z;	
}	

Εικόνα 3

Η παραγωγή των τετράδων γίνεται σταδιακά καθώς ο συντακτικός αναλυτής αναγνωρίζει τις διάφορες δομές του προγράμματος. Για παράδειγμα, κατά την ανάλυση μιας εντολής εκχώρησης, ο **parser** καλεί τη μέθοδο **emit** της κλάσης **IntermediateCode** για να προσθέσει την αντίστοιχη τετράδα στη λίστα. Αντίστοιχα, κατά την ανάλυση εκφράσεων, κάθε πράξη παράγει μια νέα τετράδα, ενώ για κάθε νέα προσωρινή τιμή που απαιτείται, χρησιμοποιείται η μέθοδος **new_temp** για τη δημιουργία μοναδικών ονομάτων προσωρινών μεταβλητών.

Τέλος, ο ενδιάμεσος κώδικας αποθηκεύεται σε αρχείο με κατάληξη **.int**, ώστε να μπορεί να χρησιμοποιηθεί ως είσοδος για το επόμενο στάδιο της μεταγλώττισης, δηλαδή τη μετατροπή σε κώδικα Assembly. Η μέθοδος **write_to_file** της κλάσης **IntermediateCode** αναλαμβάνει την αποθήκευση όλων των τετράδων σε μορφή ευανάγνωστη για τον χρήστη ή για περαιτέρω επεξεργασία.

2.4 Πίνακας Συμβόλων

Ας περάσουμε τώρα στον πίνακα συμβόλων όπου στην παρούσα υλοποίηση ο υλοποιείται μέσω των κλάσεων **Symbol** και **SymbolTable**, παρέχοντας μια ολοκληρωμένη και ευέλικτη διαχείριση των μεταβλητών και των χαρακτηριστικών τους σε όλα τα επίπεδα εμβέλειας (**scopes**) του προγράμματος.

Η κλάση **Symbol** αναπαριστά κάθε σύμβολο που μπορεί να συναντήσει ο μεταγλωττιστής, όπως μεταβλητές, παραμέτρους ή άλλα αναγνωριστικά. Κάθε αντικείμενο **Symbol** περιέχει πληροφορίες όπως το όνομα του συμβόλου, το είδος του (π.χ. μεταβλητή), τον τύπο του (στην παρούσα γλώσσα, ακέραιος), το επίπεδο εμβέλειας στο οποίο ανήκει και τη διεύθυνση μνήμης που του έχει αποδοθεί. Η μέθοδος **__repr__** εξασφαλίζει ότι κάθε σύμβολο μπορεί να παρουσιαστεί με ευανάγνωστο τρόπο, διευκολύνοντας τον έλεγχο και την αποσφαλμάτωση.

Η κλάση **SymbolTable** υλοποιεί τη βασική δομή του πίνακα συμβόλων ως μια στοίβα από λεξικά, όπου κάθε λεξικό αντιστοιχεί σε ένα επίπεδο εμβέλειας (**scope**). Με αυτόν τον τρόπο, υποστηρίζεται εύκολα η διαχείριση τοπικών και καθολικών μεταβλητών,

καθώς και η είσοδος και έξοδος από νέα **scopes** κατά την ανάλυση δομών όπως συναρτήσεις, διαδικασίες ή μπλοκ εντολών. Η μέθοδος **enter_scope** προσθέτει ένα νέο, κενό λεξικό στη στοίβα, σηματοδοτώντας την είσοδο σε νέο **scope**, ενώ η **exit_scope** αφαιρεί το πιο εσωτερικό **scope** όταν ολοκληρώνεται η ανάλυση του αντίστοιχου μπλοκ.

Στη συνέχεια, η δήλωση νέου συμβόλου γίνεται μέσω της μεθόδου **declare**, η οποία ελέγχει αν το όνομα έχει ήδη δηλωθεί στο τρέχον επίπεδο εμβέλειας (προλαμβάνοντας διπλές δηλώσεις) και στη συνέχεια δημιουργεί το αντίστοιχο αντικείμενο **Symbol**, αποδίδοντάς του μια μοναδική διεύθυνση μνήμης εντός του **scope**. Η μέθοδος **lookup** επιτρέπει την αναζήτηση ενός συμβόλου, ξεκινώντας από το πιο εσωτερικό **scope** και προχωρώντας προς τα έξω, διασφαλίζοντας έτσι τη σωστή ορατότητα και χρήση των μεταβλητών σύμφωνα με τους κανόνες της γλώσσας. Σε περίπτωση που ένα σύμβολο δεν βρεθεί, προκαλείται άμεσα συντακτικό σφάλμα.

Τέλος, ο πίνακας συμβόλων μπορεί να αποθηκευτεί σε αρχείο με τη μέθοδο **write_to_file**, παρέχοντας μια πλήρη και οργανωμένη αναφορά όλων των συμβόλων που χρησιμοποιήθηκαν στο πρόγραμμα, μαζί με τα χαρακτηριστικά τους και το επίπεδο εμβέλειας στο οποίο ανήκουν.

Λίαν συντόμως.

Για την κλάση **SymbolTable** αυτές είναι οι συναρτήσεις:

__init__(self)	Αρχικοποιεί τον πίνακα συμβόλων. Δημιουργεί μια στοίβα από λεξικά (ένα για κάθε scope) και έναν μετρητή διευθύνσεων για κάθε επίπεδο εμβέλειας
enter_scope(self)	Εισέρχεται σε νέο επίπεδο scope, προσθέτοντας ένα νέο, κενό λεξικό στη στοίβα scopes και μηδενίζοντας τον μετρητή διευθύνσεων για το νέο scope
exit_scope(self)	Εξέρχεται από το τρέχον scope, αφαιρώντας το πιο εσωτερικό λεξικό και τον αντίστοιχο μετρητή διευθύνσεων
declare(name, kind, typ='integer')	Δηλώνει νέο σύμβολο στο τρέχον scope. Ελέγχει για διπλή δήλωση, δημιουργεί αντικείμενο Symbol και του αποδίδει μοναδική διεύθυνση μνήμης
lookup(name)	Αναζητά ένα σύμβολο με το δοσμένο όνομα ξεκινώντας από το πιο εσωτερικό

	score προς τα έξω. Επιστρέφει το σύμβολο αν βρεθεί, αλλιώς εγείρει σφάλμα για αδήλωτο αναγνωριστικό
write_to_file(filename)	Αποθηκεύει τον πίνακα συμβόλων σε αρχείο, γράφοντας αναλυτικά τα σύμβολα κάθε score και τα χαρακτηριστικά τους

Για την κλάση **Symbol** αυτές είναι οι συναρτήσεις:

def __init__(self, name, kind, typ, scope_level, address	Αρχικοποιεί το αντικείμενο σύμβολο με το όνομα, το είδος, τον τύπο, το επίπεδο εμβέλειας και τη διεύθυνση μνήμης
__repr__(self)	Επιστρέφει αναγνώσιμη αναπαράσταση του συμβόλου

Ενότητα 3^η: Παραγωγή Τελικού Κώδικα

Η παραγωγή του τελικού κώδικα αποτελεί το τελευταίο στάδιο της διαδικασίας μεταγλώττισης και έχει ως στόχο τη μετατροπή του ενδιάμεσου κώδικα (τετράδων) σε κώδικα **Assembly** για την αρχιτεκτονική **RISC-V**. Η διαδικασία αυτή υλοποιείται στην κλάση **RiscVGenerator**, η οποία λαμβάνει ως είσοδο τη λίστα των τετράδων που έχουν παραχθεί από το προηγούμενο στάδιο και παράγει ως έξοδο μια ακολουθία εντολών **Assembly**, κατάλληλων για εκτέλεση σε προσομοιωτή ή πραγματικό επεξεργαστή **RISC-V**.

Η βασική μέθοδος της κλάσης **RiscVGenerator** είναι η **generate**, η οποία επεξεργάζεται σειριακά κάθε τετράδα του ενδιάμεσου κώδικα. Για κάθε τετράδα, ο μεταγλωττιστής επιλέγει το κατάλληλο μοτίβο μετάφρασης, ανάλογα με τον τελεστή της (π.χ. εκχώρηση, αριθμητική πράξη, σύγκριση, εντολή εισόδου/εξόδου). Για παράδειγμα, μια εκχώρηση τιμής μεταφράζεται σε εντολές φόρτωσης και αποθήκευσης (**li** και **sw**), ενώ μια πρόσθεση μεταφράζεται σε εντολές φόρτωσης των **τελεστών** (**lw**), εκτέλεσης της **πράξης** (**add**) και αποθήκευσης του αποτελέσματος. Οι συγκριτικές πράξεις (π.χ. **<**, **>**, **=**, **<>**) μετατρέπονται σε εντολές διακλάδωσης (**beq**, **bne**, **blt**, **bge**, κ.λπ.), με τη δημιουργία κατάλληλων **ετικετών** (**labels**) για την υλοποίηση της ροής ελέγχου.

Ιδιαίτερη προσοχή δίνεται στο κομμάτι της διαχείρισης τόσο των προσωρινών μεταβλητών όσο και των **labels**, ώστε να διασφαλίζεται ότι κάθε τιμή αποθηκεύεται και ανακτάται με ακρίβεια κατά την εκτέλεση του παραγόμενου κώδικα. Η μέθοδος **load** χρησιμοποιείται για να φορτώνει μια τιμή, είτε αυτή είναι σταθερά είτε βρίσκεται ήδη στη μνήμη, σε κάποιον καταχωρητή της αρχιτεκτονικής **RISC-V**. Παράλληλα, κάθε φορά που δημιουργείται μια νέα εντολή **Assembly**, η μέθοδος **emit** την προσθέτει στη λίστα εξόδου και της αποδίδει ένα μοναδικό **label**. Αυτός ο μηχανισμός με τα **labels** διευκολύνει σημαντικά τόσο την παρακολούθηση της ροής του προγράμματος όσο και την αποσφαλμάτωση, αφού επιτρέπει την εύκολη αναγνώριση και διαχείριση των σημείων διακλάδωσης και των εντολών που σχετίζονται με τη ροή ελέγχου.

Οι εντολές εισόδου/εξόδου (όπως η γράψε) μεταφράζονται σε κατάλληλες κλήσεις συστήματος (**ecall**) της **RISC-V**, με τη χρήση των καταχωρητών **a0** και **a7** για τη μεταφορά των παραμέτρων και του κωδικού λειτουργίας. Η τελική λίστα εντολών **Assembly** αποθηκεύεται σε αρχείο με κατάληξη **.asm** μέσω της μεθόδου **write_to_file**, επιτρέποντας την άμεση χρήση του παραγόμενου κώδικα σε προσομοιωτή ή πραγματικό περιβάλλον RISC-V.

Η μεταγλώττιση των τετραδών (ή τριάδων, δηλαδή του ενδιαμέσου κώδικα τριών διευθύνσεων) σε τελικό κώδικα **Assembly** είναι ένα στάδιο-γέφυρα ανάμεσα στη λογική δομή του προγράμματος και στη χαμηλού επιπέδου εκτέλεση στη γλώσσα στόχο, όπως η **RISC-V**. Ακολουθεί η αναλυτική διαδικασία μετατροπής των τετράδων σε τελικό κώδικα **Assembly**.

Βήμα 1: Ανάγνωση και Επεξεργασία Τετραδών

Ο κώδικας ξεκινά με την ανάγνωση της λίστας των τετραδών που έχουν παραχθεί από το στάδιο του ενδιαμέσου κώδικα. Κάθε τετράδα έχει τη μορφή (**op**, **arg1**, **arg2**, **res**), όπου:

- **op**: ο τελεστής (π.χ. '+', ':=', '>', 'write')
- **arg1**, **arg2**: τα ορίσματα (π.χ. μεταβλητές, αριθμοί, προσωρινές τιμές)
- **res**: το αποτέλεσμα ή η μεταβλητή προορισμού

Βήμα 2: Αναγνώριση Τύπου Τετράδας

Για κάθε τετράδα, ο μεταγλωττιστής προσδιορίζει τον τύπο της εντολής:

- Αν είναι εκχώρηση (:=)
- Αν είναι αριθμητική πράξη (+, -, *, /)
- Αν είναι σύγκριση (>, <, >=, <=, =, <>)
- Αν είναι εντολή εισόδου/εξόδου (**write**)
- Αν είναι ειδική εντολή μπλοκ (**begin_block**, **end_block**)

Βήμα 3: Μετάφραση σε Assembly

Για κάθε τύπο τετράδας, εφαρμόζεται το κατάλληλο μοτίβο μετάφρασης:

- Εκχώρηση (:=)

1. Φόρτωση της τιμής του **arg1** σε καταχωρητή (**t0**) με τη μέθοδο **load**.
2. Αποθήκευση του καταχωρητή στη θέση μνήμης που αντιστοιχεί στο **res** (**sw t0, res**).

- Αριθμητική πράξη (+, -, *, /)

1. Φόρτωση των ορισμάτων **arg1** και **arg2** σε καταχωρητές (**t1, t2**).
2. Εκτέλεση της πράξης με τη σωστή εντολή **Assembly** (**add, sub, mul, div**).
3. Αποθήκευση του αποτελέσματος στον καταχωρητή (**t3**) και στη συνέχεια στη θέση μνήμης του **res**.

- Σύγκριση

1. Φόρτωση των ορισμάτων σε καταχωρητές.
2. Δημιουργία εντολής διακλάδωσης (**ble, bge, blt, bgt, beq, bne**) που, αν δεν ισχύει η σύγκριση, μεταφέρει τη ροή σε ειδικό **label** (**SKIP_res**).

3. Καταγραφή του **label** για μελλοντική χρήση (π.χ. στο τέλος του μπλοκ).

- **Εντολή write**

1. Φόρτωση της τιμής προς εκτύπωση στον καταχωρητή **a0**.

2. Ρύθμιση του καταχωρητή **a7** με τον κωδικό λειτουργίας για εκτύπωση.

3. Εκτέλεση της εντολής συστήματος (**ecall**).

- **Εντολές μπλοκ**

1. Εισαγωγή σχολίου για την αρχή ή το τέλος του μπλοκ.

2. Τοποθέτηση των απαραίτητων **labels** για τη ροή ελέγχου.

Βήμα 4: Διαχείριση Προσωρινών Μεταβλητών και Labels

- Για κάθε ενδιάμεσο αποτέλεσμα, χρησιμοποιούνται προσωρινές μεταβλητές (π.χ. **T_0**, **T_1**), οι οποίες τοποθετούνται σε συγκεκριμένες θέσεις μνήμης.
- Για κάθε διακλάδωση, δημιουργείται ένα μοναδικό **label** (**SKIP_T_2**, **L5** κ.λπ.), το οποίο βοηθά στη διατήρηση της σωστής ροής του προγράμματος.

Βήμα 5: Παραγωγή και Αποθήκευση Τελικού Κώδικα

Κάθε παραγόμενη εντολή **Assembly**, μαζί με το αντίστοιχο **label**, προστίθεται στη λίστα εξόδου μέσω της μεθόδου **emit**. Στο τέλος, όλες οι εντολές γράφονται σε αρχείο με κατάληξη **.asm** με τη μέθοδο **write_to_file**. Παρακάτω απεικονίζεται ένα παράδειγμα μετατροπής ενδιάμεσου κώδικα σε **Assembly**.

1: **begin_block** main __

2: := 10 _a

3: := 25 _b

4: := 17 _c

5: := a _μέγιστο

6: > b μέγιστο T_0

7: := b _μέγιστο

8: > c μέγιστο T_1

9: := c _μέγιστο

10: **write** μέγιστο __

11: **end_block** main __

L1: # **begin_block** main

L2: li t0, 5

L3: sw t0, a

L4: li t0, 10

L5: sw t0, b

L6: lw t1, a

L7: lw t2, b

L8: # if not (a < b) skip block

L9: bge t1, t2, SKIP_T_0

L10: SKIP_T_0:

L11: lw t0, a

L12: sw t0, c

L13: lw t0, b

L14: sw t0, c

L15: # **end_block** main